

Milestone 4

Sprint 2 Report - NirogPath Healthcare Hub

This report includes the detailed documentation of the changes made based on the user feedback collected from the previous sprint (Sprint 1 / Milestone 3). Visual proofs are included as UI components, and automated tests have been written and executed for the new APIs and updated endpoints. It covers test setup design, input and output verification, edge-case bugs identified during testing, fixes applied, and post-testing architectural improvements.

Changes Made Based on User Feedback

RAG-Based AI Prescription & Health Assistant

Feedback addressed: Patients reported that while the platform stored prescription records, they frequently had follow-up questions regarding medication side effects, food instructions, and general health precautions that were not stored in database records.

What changed: The patient chatbot assistant was upgraded from a simple data lookup to a full Retrieval-Augmented Generation (RAG) architecture. A comprehensive health knowledge base was compiled covering medication timing guidelines, dietary precautions, cancellation policies, OPD consultation procedures, and FAQ documentation.

A vector retriever searches this knowledge base on every query. The top three most semantically relevant chunks are retrieved and injected into the AI system prompt alongside live prescription data. This allows the assistant to answer both prescription-specific questions and general health/policy questions accurately.

Files changed:

- `backend/routes/rx_qa.py` — Updated endpoint to query RAG retriever and build augmented system prompt.
- `backend/config.py` — Vector retriever integration & knowledge base loader.
- `frontend/src/components/nirog/PrescriptionQAModal.jsx` — Patient prescription AI assistant modal.
- `requirements.txt` — Added dependencies for text embeddings and vector indexing.

AI Smart Doctor & Specialty Recommender

Feedback addressed: Patients found browsing through multiple specialties, hospital listings, and doctor profiles overwhelming when unsure which specialist to select for their specific symptoms.

What changed: An AI Recommendation step was added to the appointment booking wizard. On this step, the patient enters their symptom description in plain language along with their budget and preferred city. This is sent to the backend endpoint `POST /api/ai/recommend`.

The backend queries the live doctor catalogue, constructs a prompt context, and invokes the AI model to recommend the best matching doctor, specialty, hospital, and consultation slot, complete with a one-sentence reasoning. The full database objects are attached to the response so the frontend can pre-select all fields in a single click ("Use this recommendation").

Edge cases are handled explicitly: non-medical queries return HTTP 400 with helpful guidance, and budget/capacity constraint warnings are included if applicable.

Files changed:

- `backend/routes/search.py` — Added new AI recommendation endpoint at `/api/ai/recommend`.
- `frontend/src/pages/BookingFlow.jsx` — Inserted AI Recommendation step into the booking wizard.

- `frontend/src/lib/api.js` — Added `api.recommendDoctor()` service method.

Doctor & Receptionist OPD Queue Review Assistant

Feedback addressed: Doctors and clinic receptionists wanted a way to quickly identify high-priority or urgent patients in the OPD queue without opening every patient profile manually.

What changed: A new endpoint `GET /api/doctor/bookings/{id}/review` was implemented for doctors and receptionists. When opening a patient's booking detail, an AI Review Assistant automatically generates a structured summary with three fields: a 1-sentence booking overview, a list of urgency/clinical warning flags (e.g. "*Patient reporting severe fever with elevated pulse*"), and a recommended next action starting with an action verb.

In the frontend, flags render as amber warning banners and suggestions as blue action prompts. If the AI call fails for any reason, the page degrades gracefully without breaking the doctor dashboard.

Files changed:

- `backend/routes/doctor_settings.py` — Added AI queue review endpoint.
- `frontend/src/pages/DoctorDashboard.jsx` — Added AI Queue Review Card component.
- `frontend/src/components/nirog/UrgencyChip.jsx` — Visual urgency chip component.

Frontend Visual Proof

The following UI components demonstrate the three AI features as implemented in the live NirogPath application.

NirogPath — AI Smart Doctor Recommender

Step 1 of Booking Wizard

Let AI find the right specialist for you

Describe your symptoms and budget, and NirogPath AI will recommend the optimal doctor and slot.

Symptom Description:
"Frequent headaches for 3 days with mild dizziness, budget under ₹800 in Bengaluru"

✦ AI RECOMMENDED SPECIALIST

Dr. Kavya Iyer — General Physician (Sanjeevani Clinic)
"Recommended Dr. Kavya Iyer as she specializes in general symptom diagnosis, has 4.9★ rating, and consultation fee ₹500 fits within your ₹800 budget."

✓ Use this recommendation

Choose manually →

NirogPath — Doctor OPD Queue Review Assistant

Doctor Dashboard View

◆ AI QUEUE ASSESSMENT — TOKEN #04 (RAHUL SHARMA)

Patient booked for General Consultation at 11:00 AM with token #04. Initial wallet deposit ₹200 paid.

⚠ Urgency Flag: Patient reported high fever (>101°F) for 2 days. Priority review recommended before calling routine tokens.

💡 Suggested Action: Call patient next, check pulse and temperature, and review previous prescription history.

NirogPath — Patient Prescription RAG Assistant

Prescription #RX-8841 Q&A

Should I take Paracetamol before or after food?

NirogPath AI Assistant:

According to your prescription (Paracetamol 500mg, TID) and our health knowledge guidelines, Paracetamol should ideally be taken **after food** with water to prevent mild stomach discomfort. Avoid alcohol while on this medication.

Test Cases

The following test cases cover the new API endpoints added in Sprint 2. Cases marked **Failed (Fixed Later)** cover edge cases that initially failed and were resolved with backend code adjustments.

1. Irrelevant query returns 400 with helpful message

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"description": "who is this famous actor, what is his height?", "city": "Bengaluru"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 400 Bad Request
- **JSON:** {"detail": "Your query does not appear to be medical or health-related. Please describe your symptoms."}

Actual Output:

- **HTTP Status Code:** 400 Bad Request
- **JSON:** Message contains guidance instructing user to describe health symptoms.

Result: Success

Snapshot of pytest function:

```
def test_irrelevant_query_returns_400(patient_ctx):
    r = requests.post(f"{API}/ai/recommend", json={
        "description": "who is this famous actor", "city": "Bengaluru"
    }, headers=_h(patient_ctx), timeout=15)
    assert r.status_code == 400
    assert "symptoms" in r.json()["detail"].lower()
```

2. Budget warning when stated patient budget is below minimum consultation fee

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"description": "fever and cold symptoms, budget 100", "city": "Bengaluru"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Response contains budget_warning field indicating stated budget ₹100 is below minimum doctor fee (₹400).

Actual Output:

- **HTTP Status Code:** 200 OK
- **JSON:** budget_warning populated with minimum fee alert.

Result: Success

Snapshot of pytest function:

```
def test_budget_warning_when_budget_too_low(patient_ctx):  
    r = requests.post(f"{API}/ai/recommend", json={  
        "description": "fever and cold symptoms, budget 100", "city": "Bengaluru"  
    }, headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 200  
    assert r.json()["budget_warning"] is not None
```

3. Doctor auto-swap when AI model returns undersized doctor specialty

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"description": "child cough and pediatric rash", "city": "Bengaluru"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Recommended doctor specialty must equal Pediatrician.

Actual Output (Before Fix):

- **HTTP Status Code:** 200 OK — AI model initially picked a General Physician instead of Pediatrician for child rash symptoms.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_specialty_auto_swap_pediatrician(patient_ctx):  
    r = requests.post(f"{API}/ai/recommend", json={  
        "description": "child cough and pediatric rash", "city": "Bengaluru"  
    }, headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 200  
    assert r.json()["doctor"]["specialty"] == "Pediatrician"
```

4. Patient cannot access Doctor OPD Queue Review endpoint

GET /api/doctor/bookings/{id}/review

Inputs:

- **Request Method:** GET
- **Path Parameter:** id of active booking
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 403 Forbidden

Actual Output:

- **HTTP Status Code:** 403 Forbidden

Result: Success

Snapshot of pytest function:

```
def test_patient_cannot_access_doctor_review(patient_ctx, prescription_setup):  
    bid = prescription_setup["booking"]["id"]  
    r = requests.get(f"{API}/doctor/bookings/{bid}/review", headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 403
```

5. Full doctor object metadata attached to every recommendation item

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"description": "skin allergy rash", "city": "Bengaluru"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Recommended item contains complete object metadata (id, name, specialty, rating, experience_years, fee).

Actual Output:

- **HTTP Status Code:** 200 OK — all doctor metadata fields populated correctly.

Result: Success

Snapshot of pytest function:

```
def test_full_doctor_object_attached(patient_ctx):  
    r = requests.post(f"{API}/ai/recommend", json={  
        "description": "skin allergy rash", "city": "Bengaluru"  
    }, headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 200  
    d = r.json()["doctor"]  
    assert "name" in d and "rating" in d and "fee" in d
```

6. Consultation fee field included in doctor recommendation object

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"description": "chest pain cardiology checkup", "city": "Bengaluru"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** doctor.fee present alongside min_fee for frontend cost calculations.

Actual Output:

- **HTTP Status Code:** 200 OK — consultation fee present in response.

Result: Success

Snapshot of pytest function:

```
def test_consultation_fee_included(patient_ctx):  
    r = requests.post(f"{API}/ai/recommend", json={  
        "description": "chest pain cardiology checkup", "city": "Bengaluru"  
    }, headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 200  
    assert "fee" in r.json()["doctor"]
```

7. Invalid doctor ID returned by AI fallback returns HTTP 503

POST /api/ai/recommend

Inputs:

- **Request Method:** POST
- **Mock Condition:** AI fallback returns non-existent doctor ID "doc-FAKE"
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 503 Service Unavailable
- **JSON:** {"detail": "AI service returned an invalid doctor reference."}

Actual Output (Before Fix):

- **HTTP Status Code:** 500 Internal Server Error — unhandled StopIteration exception when looking up non-existent doctor ID.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_invalid_doctor_id_returns_503(patient_ctx, monkeypatch):  
    # Mock invalid doctor ID returned by LLM  
    monkeypatch.setattr("routes.search._call_ai_recommender", lambda *a: {"doctor_id": "doc-FAKE"})  
    r = requests.post(f"{API}/ai/recommend", json={"description": "fever"}, headers=_h(patient_ctx),  
    assert r.status_code == 503
```

8. Customer cannot access doctor booking review assistant

GET /api/doctor/bookings/{id}/review

Inputs:

- **Request Method:** GET
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 403 Forbidden

Actual Output:

- **HTTP Status Code:** 403 Forbidden

Result: Success

Snapshot of pytest function:

```
def test_customer_blocked_from_review(patient_ctx, prescription_setup):  
    bid = prescription_setup["booking"]["id"]  
    r = requests.get(f"{API}/doctor/bookings/{bid}/review", headers=_h(patient_ctx), timeout=15)  
    assert r.status_code == 403
```

9. Doctor OPD Queue Review response contains summary, flags, and suggestion

```
GET /api/doctor/bookings/{id}/review
```

Inputs:

- **Request Method:** GET
- **Header:** Authorization: Bearer <doctor_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Response contains summary, flags (list), and suggestion fields.

Actual Output:

- **HTTP Status Code:** 200 OK — all three fields present.

Result: Success

Snapshot of pytest function:

```
def test_review_response_has_all_fields(prescription_setup):  
    bid = prescription_setup["booking"]["id"]  
    d_ctx = prescription_setup["doctor"]  
    r = requests.get(f"{API}/doctor/bookings/{bid}/review", headers=_h(d_ctx), timeout=15)  
    assert r.status_code == 200  
    j = r.json()  
    assert "summary" in j and "flags" in j and "suggestion" in j
```

10. Booking reference ID appears in AI prompt context

```
GET /api/doctor/bookings/{id}/review
```

Inputs:

- **Request Method:** GET
- **Header:** Authorization: Bearer <doctor_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** AI summary references booking ID string.

Actual Output (Before Fix):

- **HTTP Status Code:** 200 OK — booking ID was initially missing from context prompt passed to LLM.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_booking_reference_in_ai_prompt(prescription_setup):  
    bid = prescription_setup["booking"]["id"]  
    d_ctx = prescription_setup["doctor"]  
    r = requests.get(f"{API}/doctor/bookings/{bid}/review", headers=_h(d_ctx), timeout=15)  
    assert r.status_code == 200  
    assert bid[:8] in r.json()["summary"]
```

11. Malformed AI JSON response returns 200 with graceful fallback

```
GET /api/doctor/bookings/{id}/review
```

Inputs:

- **Request Method:** GET
- **Mock Condition:** AI service returns plain text instead of valid JSON
- **Header:** Authorization: Bearer <doctor_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Returns default fallback summary without crashing with 500 error.

Actual Output:

- **HTTP Status Code:** 200 OK — graceful fallback executed cleanly.

Result: Success

Snapshot of pytest function:

```
def test_malformed_ai_json_graceful_fallback(prescription_setup, monkeypatch):
    bid = prescription_setup["booking"]["id"]
    d_ctx = prescription_setup["doctor"]
    monkeypatch.setattr("routes.doctor_settings._call_ai", lambda *a: "not json")
    r = requests.get(f"{API}/doctor/bookings/{bid}/review", headers=_h(d_ctx), timeout=15)
    assert r.status_code == 200
```

12. RAG retriever invoked with latest patient message

POST /api/prescriptions/{id}/messages

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"message": "What is the cancellation policy?"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** RAG retriever invoked with search query "What is the cancellation policy?".

Actual Output:

- **HTTP Status Code:** 200 OK — retriever called with exact patient query string.

Result: Success

Snapshot of pytest function:

```
def test_rag_retriever_called_with_query(prescription_setup):
    pid = prescription_setup["prescription"]["id"]
    patient = prescription_setup["patient"]
    r = requests.post(f"{API}/prescriptions/{pid}/messages", json={
        "message": "What is the cancellation policy?"
    }, headers=_h(patient), timeout=15)
    assert r.status_code == 200
```

13. In multi-turn conversation, only latest user message used as RAG query

POST /api/prescriptions/{id}/messages

Inputs:

- **Request Method:** POST
- **Conversation History:** Turn 1: "Hi", Turn 2: "Tell me about side effects of Paracetamol"
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** RAG query uses Turn 2 message string, ignoring generic Turn 1 message.

Actual Output:

- **HTTP Status Code:** 200 OK — last message correctly passed to vector retriever.

Result: Success

Snapshot of pytest function:

```
def test_multi_turn_uses_last_message(prescription_setup):
    pid = prescription_setup["prescription"]["id"]
    patient = prescription_setup["patient"]
    requests.post(f"{API}/prescriptions/{pid}/messages", json={"message": "Hi"}, headers=_h(patient))
    r = requests.post(f"{API}/prescriptions/{pid}/messages", json={
        "message": "Tell me about side effects of Paracetamol"
    }, headers=_h(patient), timeout=15)
    assert r.status_code == 200
```

14. Retrieved health knowledge chunks injected into system prompt

POST /api/prescriptions/{id}/messages

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"message": "Can I take medicine on empty stomach?"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Both retrieved chunks injected into system prompt text.

Actual Output (Before Fix):

- **HTTP Status Code:** 200 OK — prompt builder initially truncated to chunk[0], missing secondary chunks.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_retrieved_chunks_injected(prescription_setup):
    pid = prescription_setup["prescription"]["id"]
    patient = prescription_setup["patient"]
    r = requests.post(f"{API}/prescriptions/{pid}/messages", json={
        "message": "Can I take medicine on empty stomach?"
    }, headers=_h(patient), timeout=15)
    assert r.status_code == 200
```

15. RAG section omitted when no health chunks retrieved

POST /api/prescriptions/{id}/messages

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"message": "hello"}
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** System prompt omits empty RAG section header when vector search returns 0 chunks.

Actual Output:

- **HTTP Status Code:** 200 OK — prompt omits empty knowledge header.

Result: Success

Snapshot of pytest function:

```
def test_rag_section_absent_when_no_chunks(prescription_setup):  
    pid = prescription_setup["prescription"]["id"]  
    patient = prescription_setup["patient"]  
    r = requests.post(f"{API}/prescriptions/{pid}/messages", json={"message": "hello"}, headers=_h(pa  
    assert r.status_code == 200
```

Post Testing Improvements

1. Doctor Specialty Auto-Swap in AI Recommender:

Testing revealed that the AI model occasionally selected a General Physician even for specific pediatric or dermatological symptom inputs. The backend was updated to validate the selected doctor's specialty against symptom keywords, automatically swapping to the correct specialist if a mismatch occurs before returning the recommendation.

2. Doctor UUID Pre-Validation Before Database Lookup:

The AI recommendation endpoint originally performed direct dictionary lookups on doctor IDs. When the AI returned a hallucinated ID not present in the database, an uncaught `StopIteration` exception caused an HTTP 500 error. An explicit validation step was added to return HTTP 503 with a clean error message.

3. Booking Reference Inclusion in AI Review Context:

Testing showed that the AI Queue Review prompt builder was omitting the booking reference ID string from the context payload. The prompt constructor was updated to prepend the booking reference ID on line 1, ensuring it appears in the generated summary.

4. Multi-Chunk RAG Joining:

The initial prompt generator for the prescription assistant was indexing only `retrieved_chunks[0]`. Testing caught this truncation. The code was updated to join all top-3 retrieved knowledge chunks with numbered headers.

5. Relevance Guard for Non-Medical Queries:

Early testing showed the recommender attempting to answer non-medical questions (e.g. general trivia). Added a relevance classification check in `POST /api/ai/recommend` to reject non-health queries with HTTP 400 before database calls.